Madagascar Freshwater Fish Conservation Platform

Project Overview

Centralized, open-source platform for Madagascar's ~79 endemic freshwater fish species — the island's most imperiled vertebrate group. Combines public species profiles, restricted ex-situ breeding coordination, field program tracking, and cross-sector networking between zoos, researchers, and hobbyist breeders.

Designed to complement (not replace) existing systems: ZIMS, IUCN Red List, FishBase, GBIF. Fills the documented gap where no platform integrates species-level data with conservation breeding coordination.

Technical Stack

- Backend: Python, Django, PostgreSQL-PostGIS
- Data standard: Darwin Core with custom extensions for breeding coordination
- API: RESTful, decoupled from frontend
- Frontend: Modern JavaScript framework (TBD by Architecture Agent)
- GIS: PostGIS for spatial data, coordinate generalization for sensitive locations
- Auth: Five-tier access model (public → researcher → coordinator → program manager → admin)

Key Entities

- **Species** — ~79 endemic freshwater fish. IUCN status, taxonomy, distribution, population trend. Families: Bedotiidae, Cichlidae, Aplocheilidae, Anchariidae, plus endemic gobies, cave fish, others.
- **Ex-situ Population** — captive populations across zoos, aquariums, and private breeders. Links to ZIMS for institutional records. Tracks genetics, demographics, breeding recommendations.
- **Field Program** — in-situ conservation projects. Surveys, habitat restoration, community management. Links to occurrence data publishable to GBIF.
- **Institution** — zoos, aquariums, research organizations, hobbyist breeding programs (CARES, Citizen Conservation). Has tier-based access.
- **User** — five access tiers determining what data is visible and editable.
- **Occurrence Record** — Darwin Core–compliant species observation. Sensitive location data restricted by tier using GBIF coordinate generalization protocols.

Access Tiers

Tier	Role	Access
1	Public (anonymous)	Species profiles, conservation status, general distribution
2	Registered Researcher	Occurrence data, published datasets, field program summaries
3	Conservation Coordinator	Sensitive locations, breeding recommendations, transfer coordination
4	Program Manager	Population genetics, studbook-level data, institutional inventory
5	Administrator	Full system access, user management, data import/export

External Integrations

- IUCN Red List API — species assessments (read, don't duplicate)

- FishBase — ecological data (read)
- GBIF — publish occurrence data via Darwin Core Archives + IPT
- ZIMS/Species360 — captive population snapshots (via institutional data-sharing)
- CARES priority lists — species conservation priority status
- SHOAL 1,000 Fishes Blueprint — global priority alignment

Sensitive Data Rules

- Wildlife location coordinates for threatened species must use coordinate generalization (GBIF sensitive species best practices) — exact coordinates only at Tier 3+
- Breeding recommendations and genetic data restricted to Tier 4+
- No PII in public-facing layers
- Captive population counts aggregated for public display, per-institution detail at Tier 3+

Development Commands

[To be filled after scaffolding — include: run server, run tests, migrate, lint, seed data]

Conventions

[To be filled by Architecture Agent — coding standards, patterns, naming conventions]

Conservation status sourcing (mirror policy)

`Species.iucn_status` is a **denormalized mirror** of the most-recent-accepted `ConservationAssessment` for that species, never an independently editable field. This exists to guarantee one source of truth for the public status badge and for Darwin Core / GBIF exports.

Rules:

- When `iucn_sync` accepts a new `iucn_official` assessment, it must update `Species.iucn_status` to match `assessment.category` (gated on the `ALLOW_IUCN_STATUS_OVERWRITE` setting, default True).
- Operators must not edit `Species.iucn_status` directly in admin. Manual status changes must go through creating a `ConservationAssessment` row (`source=manual_expert` once that source is added) with assessor, date, and reasoning — the mirror then picks up the change via the same path.
- A species with no `iucn_taxon_id` and no manual assessment keeps `iucn_status=NULL` ; the public profile renders "Not yet assessed" rather than a stale category.
- If a manual assessment disagrees with an incoming IUCN assessment, the conflict must be flagged for review (see the Conservation Status Governance spec) — the sync must never silently overwrite a human-reviewed category without surfacing the divergence.

No foot-guns: if you find yourself reaching for `species.iucn_status = "..."` outside the mirror path, stop and route through a `ConservationAssessment` instead.

Auth (Gate 11)

NextAuth v4 with the Credentials provider, against Django's existing `/api/v1/auth/*` endpoints. Behind `NEXT_PUBLIC_FEATURE_AUTH` so the entire auth surface can be hidden from the public site by flipping one Vercel env var. Specs: `docs/planning/architecture/auth-c-d.md` , `docs/planning/specs/gate-11-auth-mvp.md` . Handover: `docs/handover/auth-gate-11-foundation.md` .

Where the moving parts live:

- `frontend/lib/auth.ts` — `authOptions`, JWT/session callbacks, the 5-min `/me/` tier refresh, `getServerDrfToken()`, `getServerTier()`.
- `frontend/app/api/auth/[...nextauth]/route.ts` — NextAuth handler.
- `frontend/middleware.ts` — flag-gated redirects for `/account` and `/dashboard/coordinator`.
- `frontend/app/{login,signup,verify,account}/` — the four auth pages.
- `frontend/components/AuthSessionProvider.tsx` — client island wrapping `<SessionProvider>` so `useSession()` works in `NavLinks` without pulling the layout into dynamic rendering.

Three rules that govern the auth code:

1. **The DRF token never reaches the browser.** It lives only on the server-side JWT. The session callback in `lib/auth.ts` projects ONLY `tier` onto `Session`. Anything you put on `Session` becomes browser-readable via NextAuth's `/api/auth/session` endpoint. Server components that need the DRF token call `getServerDrfToken()`, which reads the JWT directly via `cookies()` + `decodeJwt`.

2. **SSR forwards tokens via `apiFetch({ authToken })`.** Pass `getServerDrfToken()` as `authToken`; the helper sets `Authorization: Token <key>`. An explicit `Authorization` header in `headers` wins (escape hatch for the existing service-token `Bearer …` path).

3. **Session-first, service-token-fallback for the coordinator dashboard.** `frontend/lib/coordinatorDashboard.ts::coordinatorHeaders()` resolves in order: session-token → `COORDINATOR_API_TOKEN` → no-Authorization. The service token is an emergency fallback, not the default. Do not re-invert that order without changing the architecture spec.

Cache-poisoning rule: any fetch that uses `authToken` MUST pass `revalidate: 0` (or be on a `force-dynamic` page). Tier-aware responses must never enter Next's shared ISR cache, since the cache key isn't authentication-aware. The map page sets `force-dynamic` and the locality fetcher passes `revalidate: 0` whenever a token is present — defense-in-depth.

Logout is dual-fire: client triggers `djangoLogoutAction()` (POSTs to Django `/auth/logout/`, deletes the DRF Token row) AND `signOut()` (clears the NextAuth cookie). A logout that only clears the cookie is a bug per BA cross-cutting; both calls must run.

i18n (Gates L1 → L4)

Multilingual platform shipped behind `NEXT_PUBLIC_FEATURE_I18N`. Four locales: `en` (default, at `/`), `fr`, `de`, `es` (path-prefixed at `/fr/`, `/de/`, `/es/`). next-intl v3 with `localePrefix: "as-needed"`. French content + UI is shipped through L4. German/Spanish ride the same infrastructure with byte-identical placeholder catalogs; translated in L5/L6.

L4 added: server-action error localization (S2), `lib/husbandry` symbolic tokens (S3), Django `gettext_lazy` sweep + FR `.po` catalog (S4–S6), `User.locale` field + PATCH endpoint (S7), branded HTML email templates + `send_translated_email()` helper (S8), account-page locale picker (S9), per-locale flag-gating tests (S10), pre-flip + rollback runbook (S11), CI lint-pockets script (S12).

Specs: `docs/planning/i18n/README.md` (locked decisions D1–D18), `docs/planning/i18n/gate-L1-framework.md` (12 user stories), `docs/planning/specs/gate-L4-i18n-french-staff.md` (12 user stories), `docs/planning/architecture/i18n-architecture.md` (technical depth). Operations:

`docs/handover/i18n-corrections-workflow.md` and `docs/operations/i18n-flag-flip-runbook.md`.

Where the moving parts live:

- `frontend/i18n/routing.ts` — `routing` config plus `Link`, `useRouter`, `usePathname`, `redirect` from `createNavigation`. Use these in client components, NOT next/navigation's raw exports — they preserve locale on internal navigation and update the `NEXT_LOCALE` cookie on locale switch.
- `frontend/i18n/request.ts` — `getRequestConfig` resolves the active locale's messages from `frontend/messages/<locale>.json`.
- `frontend/middleware.ts` — composed middleware: next-intl runs first for locale negotiation, then the auth gate runs against the post-rewrite path.
- `frontend/messages/{en,fr,de,es}.json` — message catalogs. en.json is the source of truth; the others are byte-identical placeholders in L1 and get translated in L2/L5/L6.
- `frontend/components/LocaleSwitcher.tsx` — flag-gated header dropdown.
- `frontend/lib/seo.ts` + `frontend/app/sitemap.ts` + `frontend/app/robots.ts` — locale-aware metadata, hreflang tags, cross-locale `xhtml:link` annotations.
- `backend/<app>/translation.py` — `django-modeltranslation` registrations. Wired on `Species` (description, ecology_notes, distribution_narrative, morphology), `Taxon` (common_family_name), and `SpeciesHusbandry` (narrative + 6 *_notes fields, added in L4).
- `backend/i18n/models.py::TranslationStatus` — per-(model, id, field, locale) review-pipeline state. Read-only admin in L1; L3 side-by-side inline edit + admin actions; L4 admin shortcut lets reviewers edit inline without bouncing into a separate admin.
- `backend/i18n/email.py::send_translated_email()` — locale-aware transactional email helper (L4 S8). Resolves locale from explicit arg → `recipient.locale` → settings.LANGUAGE_CODE; renders `{template}_subject.txt` + `_body.txt` + (optional) `_body.html` inside `translation.override(locale)`.
- `backend/i18n/templates/email/base.html` — shared branded layout for transactional emails. Inline-styled (email-client compat), 600px responsive table, header band + body region + footer.
- `backend/locale/<locale>/LC_MESSAGES/django.po` — gettext catalogs. `.po` is source-of-truth; `.mo` is built at image-build time (`compilemessages` runs in `backend/Dockerfile`). `.mo` is gitignored.
- `backend/accounts/models.py::User.locale` — preferred-locale field (L4 S7). Captured from `request.LANGUAGE_CODE` on signup; updatable via `PATCH /api/v1/auth/me/locale/`. Drives email locale and serves as default UI locale for logged-in users.

Five rules that govern i18n code:

1. **Never use `next/link` for internal links.** Always import `{ Link }` from `@/i18n/routing`. Plain `next/link` strips the active locale on navigation. Same goes for `useRouter` and `usePathname` — use the locale-aware versions.

2. **No hardcoded English in `frontend/app/` or `frontend/components/`.** Every visible string goes through `t()` (server: `getTranslations`; client / either: `useTranslations`). `pnpm i18n:check` verifies key parity across locales in CI; add new keys to en.json AND each placeholder catalog or the check fails.

3. **Server-action errors, `lib/husbandry` helpers, and Django-side error messages are localized as of L4.** Server actions use `getTranslations("namespace")` (see `frontend/app/[locale]/signup/ actions.ts` for the pattern). `lib/husbandry` returns symbolic tokens (`DifficultyFactorToken`, `TeaserChipToken`, `TeaserSentenceToken`);

consumer components resolve them via `t(\` namespace.${token}`) . Django strings are wrapped with` gettext_lazy as `_ . **Don't introduce new hardcoded English in these pockets** –` pnpm i18n:lint-pockets` (CI) catches it.

4. **`Species.iucn_status` mirror policy still applies (see "Conservation status sourcing").** Modeltranslation's `description_<locale>` columns are independent from the IUCN mirror; conservation-status editing must still go through `ConservationAssessment`.

5. **Locale + auth = double-cache hazard.** Any tier-aware fetch already needs `revalidate: 0` per the auth rules above. Locale adds another dimension: `next-intl` middleware patches `Vary: Accept-Language` so per-locale variants don't collide. If you add a route that does both auth-gated fetches and locale-aware rendering, the existing `revalidate: 0` discipline covers it; just don't introduce locale-aware caching that ignores `Accept-Language`.

The `NEXT_PUBLIC_FEATURE_I18N` flag and per-locale flags (`NEXT_PUBLIC_FEATURE_I18N_FR/DE/ES`) gate the `<LocaleSwitcher />` visibility and the per-locale entries in the dropdown. The locale prefix routes (`/fr/...` etc.) work regardless — they're served by next-intl middleware. This is intentional in L1: the catalogs are English placeholders, so visible French URLs don't leak unfinished translations. L2 flips `..._FR=true` after French content is human- approved through the L3 review pipeline.

DeepL API key for the L3 MT pipeline lives in root `.env` as `DEEPL_API_KEY` (free tier keys end in `:fx`, paid don't). Not used in L1.

Active Initiatives

Multi-gate initiatives have their own planning hub under `docs/planning/<initiative>/README.md`. Read the hub first in any new session touching that work — it holds the locked-in decisions, gate split, and open questions so sessions can resume without backtracking.

- **Registry redesign** — `docs/planning/registry-redesign/README.md`. Porting `docs/design.md` into the production codebase. Three gates: visual system (Gate 1, pre-June 1), schema expansion (Gate 2), ex-situ coordinator dashboard (Gate 3).

Planning Documents

Planning artifacts live in `docs/planning/`:

- `architecture/` — Architecture Agent proposals
- `business-analysis/` — BA Agent assessments
- `specs/` — PM Agent gate specs with acceptance criteria
- `<initiative>/` — multi-gate initiative hubs (see Active Initiatives above)

Ideation documents live in `docs/ideation/`:

- `extinction-crisis-report.md` — domain context and species data
- `data-infrastructure-gap.md` — gap analysis and platform rationale

Git Workflow

All file changes — by agents and by the orchestrator — must be made on a feature branch, never directly on `main`.

Before writing or editing any file:

1. Check the current branch (`git branch --show-current`). If on `main` , create and check out an appropriate branch first.
2. Name branches by type and topic:
   - Planning docs: `docs/<topic>` (e.g., `docs/ba-assessment-updates`)
   - Implementation gates: `gate/<number>-<description>` (e.g., `gate/02-data-model`)
   - Features: `feat/<description>`
   - Fixes: `fix/<description>`

No agent or session should commit directly to `main` . `main` receives merges only after human review.

Agent Delegation

This project uses specialized agents in `.claude/agents/` . Delegate as follows:

Planning (before writing code)

- **@architecture** — when starting a new project or major feature, or when a technical design decision needs to be made before requirements can be finalized
- **@business-analyst** — when evaluating whether a feature belongs, analyzing requirements, or writing acceptance criteria. The BA evaluates WHAT to build.
- **@product-manager** — when breaking approved requirements into stories, defining gates and milestones, or sequencing work. The PM plans HOW to build it.
- **@ux-reviewer** — when validating that a proposed feature or flow is usable, or when reviewing interaction design
- **@conservation-writer** — when drafting or revising public-facing copy for the platform: species profile narratives, IUCN/CARES glossaries, About page, dashboard captions, empty-state and error microcopy, funder summaries, handover docs. Voice-aligns across surfaces. Not for dev-facing documentation or marketing copy.

Verification (after writing code, at gate checkpoints)

- **@test-writer** — to write adversarial tests from PM specs at each gate. The test writer works from acceptance criteria, not from the implementation.
- **@security-reviewer** — to review code for vulnerabilities. Always invoke for changes touching auth, permissions, data access, or API endpoints.
- **@code-quality-reviewer** — to review code for maintainability, pattern adherence, and readability.

Workflow

The standard flow is: Architecture → BA → PM → [you implement with unit tests] → Test Writer + Reviewers → address feedback or escalate to planning.

Human gates exist between planning layers. At designated gates during implementation, pause and invoke the verification agents before proceeding.

Planning documents live in `docs/planning/` :

- `architecture/` — Architecture Agent output
- `business-analysis/` — BA Agent output
- `specs/` — PM Agent output (gate specs with acceptance criteria)